announce-kinds: acceptance repair of 2026-10-01

announce-reactive, branch `feat/announce-kinds`, commit `2a420e2b`, 2026-10-02. Companion to `doc/announce-management-context.md` (sections 4 and 8) and `db-reactive/sql/README.md`.

1. Summary

The announce-kinds task (one Announce aggregate with the kinds GOODS, JOB and EVENT, read-model tables fed by projections, search projection, REST discriminator) was implemented on 2026-09-22 and merged into `announce-jo-branch` on 2026-09-23. Its task block in `CLAUDE.md` has not changed since.

When the worktree was reopened on 2026-10-01, one of the two acceptance commands no longer passed: `mvn -pl db-reactive test` failed on a single test. The cause was not in the announce-kinds code. The English reseed of 2026-09-28 left a table empty that the test relied on. This commit repairs that test and closes two small leftovers of the original task.

- **Stock test** in `db-reactive`: no longer depends on seed rows. It creates and removes its own row.
- **Featured item table**: `announce_featured_item.goods_id int(11)` becomes `announce_id varchar(36)`, with an index and a guarded migration for existing databases.
- **Skipped integration test** in `announce-management`: renamed so that `mvn test` runs it.
- **Doc**: `doc/announce-management-context.md` updated for the two points above.

Result after a clean build: both acceptance suites pass (section 7). The commit is on `feat/announce-kinds`, one commit ahead of `announce-jo-branch`. It is not merged and not pushed.

2. Starting point

The branch was at `05cc5114`, an ancestor of `announce-jo-branch`, five commits behind it and with no commits of its own. The baseline on that commit, after `mvn clean`:

Suite	Run	Failed	Skipped	Note
`db-reactive`	70	1	0	`CustomReactiveMappersTest.stockMovesUpAndDownAndRefusesToGoNegative`: expected not null
`announce-management`	55	0	1	the skipped test needs the real dba feed file in the import directory

Before any edit the branch was fast-forwarded to `announce-jo-branch` (`fbfc72f4`), which brought in the English seed, the reindex trigger and the newer end-to-end test. The file `db-reactive/test.sql` showed as modified, but only its dump timestamp differed. `DbUtilTest` rewrites that file on every test run, so the change was discarded before the fast-forward and again after the final run.

3. Change 1: the stock test brings its own row

Cause. The test read the first row of `announce_goods_product` and moved its stock up and down through `GoodsProductMapper.addStock` and `reduceStock`. The litemall seed provided such rows. The English seed generator (`db-reactive/sql/gen_seed_en.py`) emits no variants, because a plain classified has none, and the in-place reseed of the live database deleted the old ones. The table is empty, on this branch and on `announce-jo-branch` alike.

Fix. In `db-reactive/src/test/java/org/linlinjava/announce/db/CustomReactiveMappersTest.java` the test inserts one product row under an announce id that no announce uses, runs the same assertions as before, and an `@AfterEach` method deletes the row by that id. The seed and the live data stay untouched.

```
private static final String STOCK_ANNOUNCE_ID = "it-stock-announce";

@Test
void stockMovesUpAndDownAndRefusesToGoNegative() {
    LitemallGoodsProduct product = new LitemallGoodsProduct();
    product.setAnnounceId(STOCK_ANNOUNCE_ID);
    product.setSpecifications(new String[] {"standard"});
    product.setPrice(new BigDecimal("10.00"));
    product.setNumber(7);
    ...
    StepVerifier.create(litemallGoodsProductMapper.insertSelective(product)).expectNext(1).verifyComplete();
    assertNotNull(product.getId());
    // addStock +5, reduceStock -5, reduceStock beyond the stock returns 0, number is back to 7
}

@AfterEach
void removeStockRows() {
    LitemallGoodsProductExample example = new LitemallGoodsProductExample();
    example.or().andAnnounceIdEqualTo(STOCK_ANNOUNCE_ID);
    litemallGoodsProductMapper.deleteByExample(example).block();
}
```

The alternative, adding a sample with variants to the seed generator, was not chosen. It would have meant regenerating both seed files and running the in-place reseed on the live database again, for the benefit of one test.

4. Change 2: the featured item is keyed by announce id

Why. Announce ids are `varchar(36)` since the announce-kinds work (subject `/announce/{id}`). The table `announce_featured_item` in `db-reactive/sql/announce_auction_chat.sql` still held `goods_id int(11)`, a reference that could not point at an announce posted through the application. No Java code and no mapper uses the table yet.

	Before	After
Column	`goods_id int(11) NOT NULL`	`announce_id varchar(36) NOT NULL`
Index	`idx_featured_window (start_time, end_time)`	the same, plus `idx_featured_announce (announce_id)`

Fresh databases get the new column from the `CREATE TABLE` statement. **Existing databases** are upgraded by section 3 of the same script, which stays re-runnable. A helper procedure renames the column only when the old one is still there, and the index is added through the existing `announce_add_index` guard:

```
CREATE PROCEDURE announce_change_column(IN p_table VARCHAR(64), IN p_old_column VARCHAR(64), IN p_ddl TEXT)
BEGIN
  IF EXISTS (SELECT 1 FROM information_schema.COLUMNS
             WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = p_table AND COLUMN_NAME = p_old_column) THEN
    SET @sql = CONCAT('ALTER TABLE `', p_table, '` CHANGE COLUMN `', p_old_column, '` ', p_ddl);
    PREPARE stmt FROM @sql; EXECUTE stmt; DEALLOCATE PREPARE stmt;
  END IF;
END$$

CALL announce_change_column('announce_featured_item', 'goods_id',
     '`announce_id` varchar(36) NOT NULL COMMENT ''announce.id (any kind)''');
CALL announce_add_index('announce_featured_item', 'idx_featured_announce', '(`announce_id`)');
```

Existing numeric ids would be kept as strings, the same convention as the goods rows that were migrated into `announce`. The helper procedures are dropped at the end of the script.

Bootstrap. `db-reactive/sql/announce_reactive_db.sql` is generated. It was rebuilt with `db-reactive/sql/build_announce_reactive_db.sh` and its diff contains only the lines above.

Live database. The script was run once against `announce_reactive_db` on localhost. The table was empty (0 rows) before the change. `SHOW CREATE TABLE` afterwards shows `announce_id varchar(36)` and both indexes, and no helper procedure is left behind. To upgrade any other existing database, run the same command:

```
mysql -uannounce -p announce_reactive_db < db-reactive/sql/announce_auction_chat.sql
```

5. Change 3: the repository test runs with the suite

`MyBatisRepositoriesIT` checks the category and brand repositories of `announce-management` against the live database. The surefire include pattern of the module matches `*Test`, `*Tests`, `Test*` and `*TestCase`, so a class ending in `IT` never ran under `mvn test`. It only ran when named explicitly with `-Dtest`.

The class is renamed `MyBatisRepositoriesTest` (same package, `goods/infrastructure/persistence/mybatis`). Nothing else changes in it. Its rows are prefixed `it-` and removed after each test. It now adds one test to the `announce-management` run and passes.

6. Files in the commit

File	Change
`db-reactive/src/test/java/.../db/CustomReactiveMappersTest.java`	stock test inserts and deletes its own product row
`db-reactive/sql/announce_auction_chat.sql`	`announce_id varchar(36)`, new index, guarded rename for existing databases
`db-reactive/sql/announce_reactive_db.sql`	regenerated from the sources
`announce-management/src/test/java/.../mybatis/MyBatisRepositoriesTest.java`	renamed from `MyBatisRepositoriesIT.java`
`doc/announce-management-context.md`	section 4: featured item row; section 8: test paragraph

Five files, 75 insertions and 16 deletions. No production Java code changed.

7. Verification

Both acceptance commands of the task block were run in one reactor after `mvn clean`, so that `announce-management` compiles against this worktree's `db-reactive` and not against a snapshot another worktree installed:

```
mvn -B -pl db-reactive,announce-management -P '!webapp' clean test
```

Suite	Run	Failed	Skipped	Note
`db-reactive`	70	0	0	`CustomReactiveMappersTest` 4 of 4
`announce-management`	57	0	1	`AnnounceKindsEndToEndTest` 7 of 7, `AnnounceTest` 11 of 11, `MyBatisRepositoriesTest` 1 of 1

- The skipped test is the import of the real dba feed. It skips itself when that file is not in the import directory.
- The end-to-end case for the `announce.v1` relay, which timed out on this machine on 2026-09-28, passed.

- The end-to-end test loads `announce_auction_chat.sql` into a fresh MySQL container, so the changed script is also proven on an empty database.
- After the run the live database held no leftover test rows: 0 rows in `announce_goods_product`, 0 categories named `it-%`.

8. Things to know afterwards

- `db-reactive/test.sql` shows as modified after every test run, because `DbUtilTest` writes a new dump timestamp into it. Restore it with `git checkout -- db-reactive/test.sql` before a merge or a fast-forward.
- Run `mvn clean` on `db-reactive` after pulling: a stale mapper XML in `target/` from a deleted table breaks the suite.
- Other tables still hold integer goods references (`announce_collect.value_id`, `announce_comment.value_id`, `announce_footprint.goods_id`). They belong to the member context and were left alone.
- Next step: merge `feat/announce-kinds` into `announce-jo-branch`. Other developers then re-run `announce_auction_chat.sql` on their own database to pick up the renamed column.